

Praxis:

19.3 Binäre Suchbäume mit BlueJ

Jetzt wird es wieder einmal Zeit, sich an den Computer zu setzen und ein bisschen praktisch zu arbeiten. Als erstes programmieren Sie sich eine Klasse **Element** oder **Knoten**, und zwar auf die einfache Weise ohne Datenkapselung:

```
public class Element
{
    int value;
    Element left, right;

    public Element(int n)
    {
        value = n;
        left = null;
        right = null;
    }

    public void show()
    {
        System.out.println(" "+value);
    }
}
```

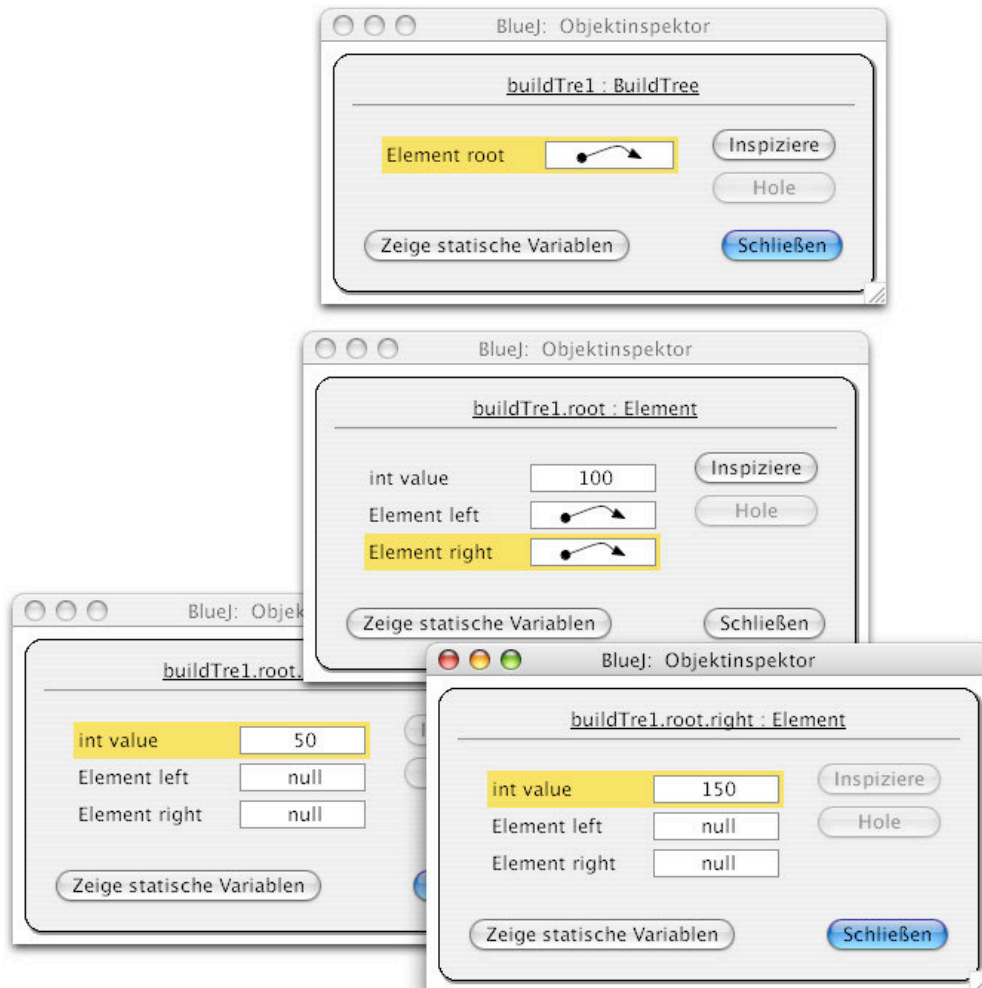
Diese Klasse speichert nur int-Zahlen. Um Objekte oder andere Datenstrukturen können wir uns später im Verlauf der Folge 19 noch kümmern, jetzt wollen wir uns auf das Wesentliche konzentrieren. Auf die Datenkapselung verzichten wir hier, um später den Zugriff auf die einzelnen Attribute wie **left** und **right** zu vereinfachen. Sonst müssten wir eigene sondierende Methoden schreiben - was an sich ja das korrekte Vorgehen wäre.

Als nächstes programmieren Sie bitte eine Klasse **BuildTree**, deren Aufgabe es ist, aus vielen solcher Elemente einen binären Suchbaum zu erstellen. Hier der vollständige Quelltext dieser Klasse:

```
public class BuildTree
{
    Element root;

    public BuildTree()
    {
        root = new Element(100);
        root.left = new Element(50);
        root.right = new Element(150);
    }
}
```

Nun schauen Sie sich mit dem **Objektinspektor** den erzeugten Baum einmal an. Der Baum wurde in der Tat genau so aufgebaut, wie es beabsichtigt wurde. Die Wurzel hat den Wert 100, der linke Nachfolger den Wert 50, und der rechte Nachfolger den Wert 150.



19-7 Ein einfacher Binärbaum wird mit dem Objektinspektor untersucht.

Übung 19.3 (2 Punkte)

Zeichnen Sie den Binärbaum, der sich ergibt, wenn folgende Methode ausgeführt wird:

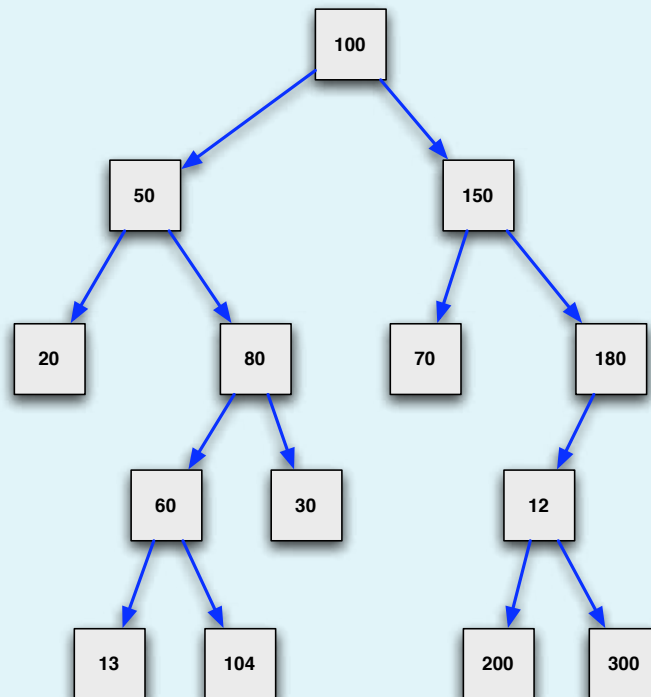
```
public BuildTree()
{
    root = new Element(100);
    root.left = new Element(50);
    root.right = new Element(150);

    root.left.left = new Element(40);
    root.left.right = new Element(45);
    root.right.left = new Element(140);
    root.right.right = new Element(165);

    Element help = root.left.right;
    help.left = new Element(17);
    help.right = new Element(35);
    help = help.left;
    help.left = new Element(10);
}
```

Übung 19.4 (2 Punkte)

Verändern Sie den Konstruktor **BuildTree()** so, dass folgender Binärbaum aufgebaut wird, und zeigen Sie mit dem Objektinspektor, dass der Baum tatsächlich erzeugt wurde.



Grundlagen:

19.4 Eine rekursive Methode zum Anzeigen der Elemente

19.4.1 Allgemeines

Ganz bewusst wird Ihnen das Programmieren einer solchen Methode nicht als Aufgabe gestellt, denn im Internet existieren bereits sehr viele Lösungen für dieses Problem, so dass dies keine echte Herausforderung für Sie mehr wäre. Wir wollen hier lieber eine bewährte Ausgabe-Methode ansehen und besprechen.

```
public void show()
{
    showR(root);
}

private void showR(Element p)
{
    if (p != null)
    {
        showR(p.left);
        p.show();
        showR(p.right);
    }
}
```

Wir sehen hier zwei Methoden, die in einer engen Beziehung stehen, die **show()**-Methode und die **showR()**-Methode. Die eigentliche Arbeit wird von der **showR()**-Methode geleistet. Das R steht für „rekursiv“. In der Tat handelt es sich um eine rekursive Methode. Die Arbeitsweise von **showR()** wird im nächsten Abschnitt näher erläutert. Vorab können wir schon einmal festhalten, dass **showR()** den gesamten binären Suchbaum in geordneter Reihenfolge durchläuft - vom kleinsten Element über die Wurzel bis zum größten Element.

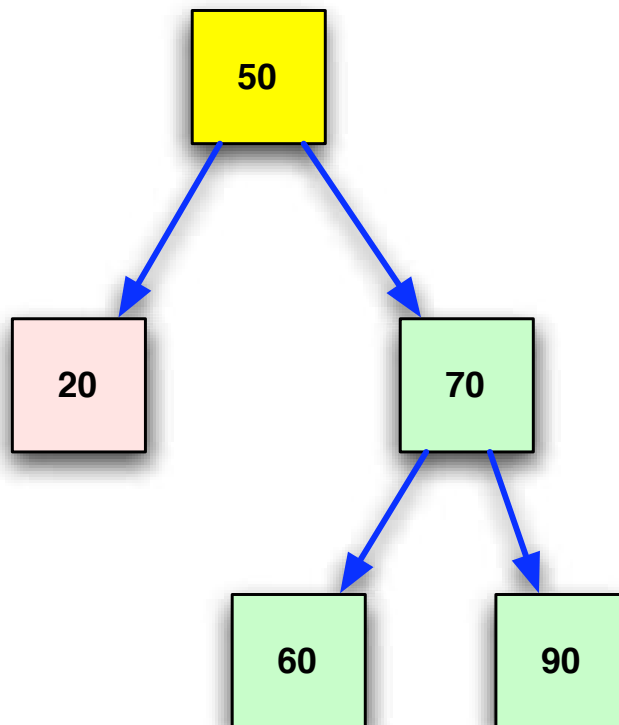
Der Nachteil von **showR()** ist die Tatsache, dass wir der Methode einen Parameter vom Typ **Element** übergeben müssen. Damit geben wir eine Information preis, die wir dem Benutzer der Klasse **BinTree** ganz im Sinne des **information hiding** eigentlich vorenthalten wollen. Aus diesem Grunde schreiben wir eine **Mantelmethode** für **showR()**, die nichts anderes tut, als **showR()** aufzurufen. Der Benutzer der Klasse **BinTree** kann nun ganz einfach **show()**-Methode aufrufen und muss sich dabei nicht mit den internen Details der Klasse **BinTree** belasten. Hätten wir **BinTree** beispielsweise mit Hilfe von Arrays implementiert, so könnte man die rekursive **showR()**-Methode sicherlich nicht mit einem Parameter vom Typ **Element** aufrufen. Beim Aufruf der Mantelmethode **show()** dagegen ist es völlig egal, wie die Klasse **BinTree** realisiert wurde.

Information hiding

Ein Konzept der objektorientierten Programmierung, eng verwandt mit dem Prinzip der **Datenkapselung**. Dem Benutzer einer Klasse werden nur die unbedingt notwendigen Informationen zugänglich gemacht, die er benötigt, um die Methoden der Klasse korrekt aufzurufen. Alle weiteren Informationen, die nicht unbedingt benötigt werden, werden vor dem Benutzer versteckt, zum Beispiel mit Hilfe von **Mantelmethoden**, die keine oder nur wenige Parameter besitzen und die eigentlichen (privaten) Methoden dann aufrufen.

19.4.2 Arbeitsweise von showR()

Die Arbeitsweise von **showR()** soll Ihnen am Beispiel des folgenden binären Suchbaums klar gemacht werden.



19-8 Ein einfacher binärer Suchbaum.

Ebene 1: Abfrage und Aufruf des linken Teilbaums

Zunächst zeigt der Zeiger `p` auf die 50. Die if-Bedingung `if (p != null)` ist also erfüllt. Nun ruft sich die Methode rekursiv selbst auf, und zwar für den linken Teilbaum von 50.

Ebene 2: Abfrage und Aufruf des linken Teilsbaums

Der Zeiger, der in Ebene 1 noch `p.left` hieß, taucht in dieser Ebene 2 als `p` wieder auf - eine Folge der Parameterübergabe. Auch hier gilt `p != null`, denn `p` zeigt auf die 20. Also wird `showR()` erneut rekursiv aufgerufen.

Ebene 3: Abfrage und Abbruch

Der Zeiger `p` (in Ebene 2 noch `p.left`) zeigt auf `null`, denn die 20 hat keinen linken Nachfolger. Damit ist die Bedingung `p != null` nicht erfüllt, und die Ebene 3 wird beendet, noch ehe sie richtig begonnen hat.

Ebene 2: Ausgabe der 20

Da die Ebene 3 aus der Ebene 2 heraus aufgerufen wurde, befinden wir uns jetzt wieder in der Ebene 2. Nachdem der erste Befehl der Ebene 2 ausgeführt wurde, nämlich der rekursive Aufruf des linken Teilbaums, wird nun der zweite Befehl ausgeführt, der `show()`-Befehl der 20. Also erscheint jetzt die Zahl 20 in der Konsole.

Ebene 2: Aufruf des rechten Teilbaums

Wir befinden uns immer noch in der Ebene 2 der Methode `showR()`. Nun wird der dritte Befehl der Methode aufgerufen. Hierbei handelt es sich um den rekursiven Aufruf von `showR()` für den rechten Teilbaum.

Ebene 3: Abfrage und Abbruch

Der Zeiger `p` (in Ebene 2 noch `p.right`) zeigt auf `null`, denn die 20 hat keinen rechten Nachfolger. Damit ist die Bedingung `p != null` nicht erfüllt, und die Ebene 3 wird abermals beendet, noch ehe sie richtig begonnen hat.

Ebene 2: Reguläres Beenden

Und wieder sind wir in der Ebene 2. Da der letzte Befehl - der rekursive Aufruf von `showR()` - abgeschlossen ist, kann die Ebene 2 beendet werden.

Ebene 1: Ausgabe der 50

Jetzt befinden wir uns wieder in der ersten Ebene des `showR()`-Befehls. Der erste Methodenaufruf wurde gerade abgearbeitet, und jetzt folgt der zweite Befehl - die Ausgabe der Zahl 50 in der Konsole.

Ebene 1: Aufruf des rechten Teilbaums

Danach wird der dritte Befehl der Methode ausgeführt, nämlich der rekursive Aufruf von `showR()` für den rechten Nachfolger von `p`.

Ebene 2: Abfrage und Aufruf des linken Teilbaums

Wir sind jetzt bei der 70. Da dieser Knoten nicht leer ist, wird mit dem linken Teilbaum der 70 weiter gemacht.

Ebene 3: Abfrage und Aufruf des linken Teilbaums

Wir sind jetzt bei der 60. Da dieser Knoten nicht leer ist, wird mit dem linken Teilbaum der 60 weiter gemacht.

Ebene 4: Abfrage und Abbruch

Der Zeiger `p` zeigt hier auf `null`, damit wird die Ebene 4 sofort wieder verlassen.

Ebene 3: Ausgabe der 60

Jetzt wird der zweite Befehl auf dieser Ebene ausgeführt, und die Zahl 60 wird angezeigt.

Ebene 3: Aufruf des rechten Teilbaums

Als nächstes kommt der dritte Befehl, nämlich der rekursive Aufruf der Methode `showR()`.

Ebene 4: Abfrage und Abbruch

Der Zeiger `p` zeigt hier auf `null`, damit wird die Ebene 4 sofort wieder verlassen.

Ebene 3: Reguläres Beenden

Alle Befehle der Ebene 3 wurden abgearbeitet, also landen wir jetzt wieder in der Ebene 2.

Ebene 2: Ausgabe der 70

Der zweite Befehl der Ebene 2 wird ausgeführt, nämlich die Ausgabe der Zahl 70.

Ebene 2: Aufruf des rechten Teilbaums

Nun kommt der dritte Befehl der Ebene 2, der rechte Teilbaum wird rekursiv aufgerufen.

Ebene 3: Abfrage und Aufruf des linken Teilbaums

Wir sind jetzt bei der 90. Da dieser Knoten nicht leer ist, wird links weiter gemacht.

Ebene 4: Abfrage und Abbruch

Der Zeiger `p` zeigt hier auf `null`, damit wird die Ebene 4 sofort wieder verlassen.

Ebene 3: Ausgabe der 90

Jetzt wird der zweite Befehl auf dieser Ebene ausgeführt, und die Zahl 60 wird angezeigt.

Ebene 3: Aufruf des rechten Teilbaums

Als nächstes kommt der dritte Befehl, nämlich der rekursive Aufruf der Methode `showRO`.

Ebene 4: Abfrage und Abbruch

Der Zeiger `p` zeigt hier auf `null`, damit wird die Ebene 4 sofort wieder verlassen.

Ebene 3: Reguläres Beenden

Alle Befehle der Ebene 3 wurden abgearbeitet, also landen wir jetzt wieder in der Ebene 2.

Ebene 2: Reguläres Beenden

Ebene 1: Reguläres Beenden

Eigentlich sollte Ihnen jetzt klar sein, wie die rekursive `showRO`-Methode funktioniert.

Übung 19.5 (2 Punkte)

Erweitern Sie die `showRO`-Methode so, dass nicht nur die Zahl selbst ausgegeben wird, sondern auch die jeweilige Rekursionsebene, so ähnlich wie es in dem ausführlichen Beispiel besprochen wurde.

Grundlagen:

19.5 Eine insert()-Methode

Wir kommen jetzt zum Höhepunkt der Folge 19. Eine **insert()**-Methode für einen binären Suchbaum zu schreiben, sollte gar nicht so schwer sein, wenn man an die Folge 17 denkt, in der wir eine **insert()**-Methode für eine doppelt verkettete Liste konstruiert haben.

Fall 1 - Der Baum ist leer

In diesem Fall ist das Einfügen eines neuen (ersten) Elements ganz einfach:

```
if (root == null)
    root = new Element(v);
```

Wenn der Baum leer ist und der **root**-Zeiger auf **null** verweist, wird einfach der Konstruktor für ein neues Element aufgerufen und die Adresse des neuen Elements in **root** gespeichert.

Fall 2 - Der Baum enthält genau ein Element

```
if (v < root.value)
    root.left = new Element(v);
else
    root.right = new Element(v);
```

Wenn die Wurzel bereits vorhanden ist, wird der neu einzufügende Wert mit der Wurzel verglichen. Ist der neue Wert kleiner als die Wurzel, so wird das neue Element als linker Teilbaum an die Wurzel angehängt, andernfalls als rechter Teilbaum.

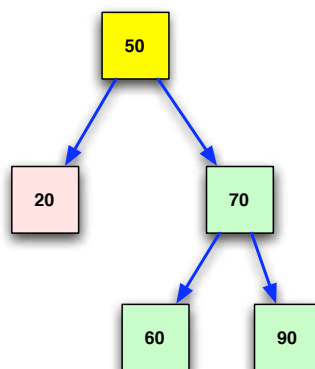
Fall 3 - Der Baum enthält viele Elemente

In Wirklichkeit enthält ein binärer Suchbaum viele Elemente. Wir erhalten eine funktionierende **insert()**-Methode, indem wir die beiden eben besprochenen Fälle geschickt kombinieren und in eine while-Schleife einbetten. Aber zunächst der Quelltext. Achten Sie auf die farbliche Kennzeichnung der Befehle!


```
public void insert(int v)
{
    if (root == null)
        root = new Element(v);
    else
    {
        Element help = root;
        while (help != null)
        {
            if (v < help.value)
                if (help.left != null)
                    help = help.left;
                else
                    { help.left = new Element(v); return; }
            else
                if (help.right != null)
                    help = help.right;
                else
                    { help.right = new Element(v); return; }
        }
    }
}
```

Wenn der Baum leer ist, wird ein neues Element angelegt und dann wird der Zeiger **root** auf dieses neue Element gesetzt (grüner Quelltext, Fall 1).

Enthält der Baum bereits Elemente, so wird ein Hilfszeiger auf die Wurzel gesetzt (**Element help = root**). Dann wird dieser Hilfszeiger so „umgebogen“, bis er auf denjenigen Knoten zeigt, an den das neue Element als linker oder rechter Teilbaum angehängt werden soll. Dies geschieht mit Hilfe einer while-Schleife. Solange der Hilfszeiger nicht den Wert null hat (solange er also auf einen Knoten zeigt), wird überprüft, ob das neue Element kleiner ist als das Element, auf das der Hilfszeiger zeigt.



19-9 Ein einfacher binärer Suchbaum.

Angenommen, der Hilfszeiger verweist gerade auf das Element 50 in Abbildung 19-9, und es soll die Zahl 30 eingefügt werden. Nun gilt die Bedingung `v < help.value`, denn `30 < 50`. Also wird nachgeschaut, ob ein linker Nachfolger der 50 existiert: `if (help.left != null) ...`. Dies ist der Fall, die 20 ist der linke Nachfolger der 50. Also wird die Anweisung `help = help.left` ausgeführt.

Der Hilfszeiger zeigt jetzt auf die 20. Dann wird die while-Schleife erneut durchlaufen. Da die 30 nicht kleiner ist als das Element 20, wird der else-Zweig dieser if-else-Anweisung ausgeführt. Hier wird zunächst einmal überprüft, ob ein rechter Nachfolger der 20 existiert. Dies ist nicht der Fall, es gibt keinen rechten Nachfolger. Also wird jetzt der Befehl `help.right = new Element(v)` ausgeführt (roter Quelltext, Fall 2). Es wird Speicherplatz für ein neues Element erzeugt und der Zeiger `help.right` wird mit diesem Speicherplatz verknüpft. Damit wird das neue Element 30 korrekt in den Baum eingefügt.

Die return-Anweisung am Ende des if- bzw. else-Zweiges ist unbedingt notwendig, weil sonst der Hilfszeiger auf das soeben eingefügte neue Element gesetzt würde. Dann würde die while-Schleife ein weiteres Mal durchlaufen und das Element ein zweites Mal in den Baum eingefügt, dann ein drittes Mal, ein viertes Mal und so weiter.

Grundlagen:

19.6 Eine rekursive insert-Methode

Eine Methode zum Einfügen von Elementen in einen binären Suchbaum haben Sie gerade kennen gelernt. Mit einer while-Schleife wird zunächst der korrekte Platz zum Einfügen des neuen Elements im Baum gesucht, und dann wird das neue Element links oder rechts an den entsprechenden Knoten angehängt.

Ganz am Anfang dieser Folge haben Sie die rekursive Definition des Begriffs Baum kennen gelernt. Genau so einfach kann man auch das Einfügen in einen binären Suchbaum rekursiv beschreiben:

Fall 1 - Der Baum ist leer

Auch hier - genau wie im nicht-rekursiven Algorithmus - ganz einfach:

```
if (tree==null)
    tree = new Element(v);
```

Fall 2 - Der Baum ist nicht leer

Ist das neue Element kleiner als die Wurzel, dann mache im linken Teilbaum weiter, ansonsten im rechten Teilbaum. Mit „mache weiter“, ist hier gemeint, dass der eben beschriebene Algorithmus auf den linken bzw. rechten Teilbaum angewandt wird.

Quelltext

Hier ist eine komplette und lauffähige rekursive **insert()**-Methode für den binären Suchbaum:

```
public void insert(int v)
{
    root = insertR(v,root);
}
```

Das war's schon? Nein, es handelt sich hier nur um die öffentliche **Mantelmethode** für die rekursive Methode **insertR()**. Die Aufgabe von **insert()** ist es, **insertR()** mit den richtigen Parametern aufzurufen, so dass der Benutzer nicht merkt, dass in Wirklichkeit eine rekursive Methode aufgerufen wird und dass diese mit Hilfe von Zeigern implementiert ist (**information hiding**).

Hier nun der Quelltext der rekursiven Einfüge-Methode:

```
private Element insertR(int v, Element tree)
{
    if (tree==null)
        tree = new Element(v);
    else if (v < tree.value)
        tree.left = insertR(v,tree.left);
    else
        tree.right = insertR(v,tree.right);
    return tree;
}
```

Quelltext-Analyse

Machen wir uns diese Methode klar. Zunächst einmal ist der Baum leer. Was geschieht, wenn wir die erste Zahl einfügen? Als Beispiel wählen wir die Zahl 100.

In der rekursiven Methode wird zunächst festgestellt, dass `tree == null` ist. Also wird ein neues Element angelegt, die Zahl 100 hineingeschrieben, und die Adresse dieses Elementes in der lokalen Zeigervariablen `tree` gespeichert:

```
tree = new Element(v);
```

Dann wird die rekursive Methode auch schon beendet. Dabei wird die Adresse, die in `tree` gespeichert ist, als Rückgabewert an die öffentliche `insert()`-Methode zurück gegeben und dort in dem Attribut `root` gespeichert.

Als nächstes soll die Zahl 50 eingefügt werden. Wieder wird von der `insert()`-Methode die rekursive `insertR()`-Methode aufgerufen. Die Bedingung `tree == null` ist nicht erfüllt, also wird im else-Zweig weitergemacht. Hier wird gefragt, ob `v < tree.value` ist. Da $50 < 100$, ist dies der Fall. Daher wird der Befehl

```
tree.left = insertR(v, tree.left);
```

ausgeführt. Dem linken Zeiger des Elementes (100) wird das Ergebnis der sondierenden Methode `insertR()` zugewiesen, das ja aus einer Speicheradresse besteht.

Um dieses Ergebnis zu ermitteln, müssen wir eine Rekursionsebene tiefer gehen. Sie kennen das ja schon von der Besprechung der rekursiven `show()`-Methode her.

Eine Ebene tiefer

In dieser tieferen Ebene hat der Parameter `tree`, der in der aufrufenden Ebene ja als `tree.left` bekannt ist, den Wert `null`. Also wird ein neues Element erzeugt, die 50 hineingeschrieben, und die Adresse dieses neuen Elementes zurück geliefert.

Wieder zurück in Ebene 1

Die erhaltene Adresse wird jetzt in `tree.left` gespeichert. Der Baum besteht jetzt aus den beiden Elementen (100) und (50), wobei (50) der linke Nachfolger von (100) ist

Als nächstes soll die Zahl 70 in dem Baum gespeichert werden. In der rekursiven Methode ist die Bedingung `tree == null` nicht erfüllt, also wird im else-Zweig weitergemacht. Hier wird gefragt, ob `v < tree.value` ist. Da $70 < 100$, ist dies der Fall. Daher wird der Befehl

```
tree.left = insertR(v, tree.left);
```

ausgeführt, und es wird eine Ebene tiefer weitergemacht.

Eine Ebene tiefer

Hier ist `tree` nicht gleich `null`, sondern zeigt auf die 50. Da $70 > 50$, wird im rechten Teilbaum weitergemacht.

Eine Ebene tiefer

Im rechten Teilbaum der (50) ist `tree == null`, daher wird hier ein neues Element erzeugt, nämlich die (70), und die Adresse dieses neuen Elementes als Parameter zurückgeliefert.

Eine Ebene höher

Diese Adresse wird nun im Zeiger `tree.right` gespeichert. Das Element (70) wird als rechter Nachfolger an das Element (50) angehängt.

Übung 19.6 (2 bzw. 3 Punkte)

Bringen Sie zunächst die rekursive Insert-Methode zum Laufen und testen Sie sie.

Erweitern Sie dann die Klasse **BinTree** um eine **memberO**-Methode, die den Wert **true** zurück liefert, wenn die gesuchte Zahl im Baum vorhanden ist. Für eine rekursive Variante bekommen Sie einen Punkt mehr, also 3 Punkte.